Dashboard Royale: Team Plan

Nov 7, 2026 · @Kevin

Summary

We answer one question for the GM: should the "bottom 10" machines really be replaced, or is it luck, location, or a leak? Everything below serves that story and the six required questions.

Key decisions:

- Use DuckDB on the Parquet files instead of MySQL. The data is small once deduplicated, already typed, and Tableau Public cannot connect to a database anyway.
- Python + DuckDB builds small, pre-aggregated "mart" tables. Tableau only ever sees those CSV/Hyper extracts.
- One model: a daily floor coin-in forecast (model chosen within Jan–Apr, tested on May–Jun) against a constant-mean baseline; the weekday average is the model, not the baseline. A July forecast refit on Jan–Jun informs staffing, promotion timing and measuring the budget actions.
- Main analytical spine: a machine scorecard that adjusts house win for theo (luck), availability (downtime), days on floor (exposure) and location.
- Budget answer is evidence-driven. We fund at most 3 action types, and "keep the money" stays on the table.

Data stack: skip MySQL, use DuckDB on Parquet

MySQL would cost us setup hours and buy us nothing we need. DuckDB gives the same SQL with zero ingestion.

Concern	MySQL + indexes	DuckDB on Parquet
Real data size	The "1 GB" is mostly the same tables twice (.parquet and .csv.gz) plus git history. Largest table is 1.3M rows	Reads the Parquet as-is; 1.3M rows aggregates in under a second on a laptop
Setup	Server install, schemas for ~25 tables, type mapping (cents, UTC timestamps, booleans), loading CSVs	`pip install duckdb`, then `SELECT … FROM 'file.parquet'`
Team sharing	Someone hosts a server and shares credentials	Each teammate runs the same scripts against their own clone
Tableau Public	Cannot connect live to a database. Public workbooks are saved as extracts from files, so we would export from MySQL anyway	Write marts straight to CSV, which Tableau Public loads
Indexing	Helps point lookups; our work is full-table aggregation	Columnar scans are already the fast path

What we keep from the MySQL idea: a proper SQL layer. DuckDB's syntax is close to standard SQL, so anyone comfortable with MySQL can write it.

Repo hygiene:

- The raw data already lives in our private team repo, so everyone clones that one repo (`git clone --depth 1` to skip history).
- Treat `data/` as read-only. Git keeps every version forever, so re-saving or modifying data files and committing them bloats the repo by hundreds of MB.
- Add `data/**/*.csv` to `.gitignore` so anyone who unzips a .csv.gz for Excel or Tableau doesn't commit the result.
- Commit code, SQL, and the small mart outputs (a few MB) that Tableau reads.

- Pin one data release (`0.9.0-draft` now). If organizers publish `1.0.0`, replace the files in a single commit and re-run the pipeline; values may shift.

Who uses which tool: Python (pandas/DuckDB, statsmodels or scikit-learn) for EDA and the model; Tableau Public (Desktop app) for the dashboard; Tableau Public for hosting, since judges need a public link.

Repo layout and pipeline

One command (`python run_all.py`) should rebuild every mart from raw data. That makes the "all code" submission and the Claims & Evidence appendix easy.

```
team-repo/
├── data/        # official data, committed once, read-only
├── sql/         # one .sql per mart, readable by judges
├── src/
│   ├── build_marts.py   # runs sql/ in DuckDB, writes marts/
│   ├── eda/             # notebooks, one per question
│   └── model/           # forecast + baseline + holdout eval
├── marts/       # small CSVs Tableau reads (committed)
├── docs/        # write-up, appendix, AI disclosure
└── run_all.py
```

Pipeline steps:

1. **Load:** DuckDB views over each Parquet file (no copy). Filter active / non-empty `location_id` in the views so nobody forgets.
2. **Validate:** row counts match the guide (56,110 daily rows, 181 days, 310 machines); `machine_hour` sums to `machine_day`; cents ÷ 100 equals the `_usd` columns.
3. **Build marts:** one table per dashboard page (list below), all money in dollars, all ratios computed as ratio of sums.
4. **Model:** reads `mart_floor_daily`, writes `mart_forecast` (date, actual, model, baseline, split), plus July 1–30 predictions from the chosen method refit on Jan–Jun.
5. **Export:** CSVs to `marts/`; Tableau Public connects to these files.

Marts to build:

Mart	Grain	Feeds
`mart_machine_scorecard`	machine, fixed Jan–Jun: z-scores, bootstrap intervals, net contribution, classification	Q1 verdict page
`mart_machine_month`	machine × month, same metrics	Q1 by period (classification stays Jan–Jun)
`mart_machine_daily`	machine × day, with zone/bank/slot of that day and machine costs	period filter, bank drill, descriptive Q1 metrics
`mart_floor_daily`	day, with machines on floor, events, weather	Q2 trends, model input (weather for EDA only)
`mart_zone_daily`	zone × day, traffic + coin-in	Q2 location demand
`mart_visits_month`	month × segment, visits and play per visit from trips	Q2 visits vs. spend
`mart_benchmark_month`	month: our coin-in vs. Missouri benchmark and LVCVA	Q2 season vs. trend
`mart_downtime_hourly`	machine × hour: availability, play, hidden-downtime flag	Q3 heatmap, hour-band rates, hidden downtime
`mart_tickets`	ticket: type, controllable flag, response and repair time, theo before and after	Q1 faults while running, Q3
`mart_player_month`	player × month (or segment × month), with expected vs. actual loss	Q4, guest drill, Q5 actual loss

Mart	Grain	Feeds
`mart_comps_tier_month`	tier × month: comp cost, actual loss, coin-in, total spend, theo, comp ratios (per $ actual loss, per $ coin-in, per $ total spend), points earned, redeemed and balance	Q5
`mart_comps_daily`	day: same columns as the tier-month mart, daily	Q5 trend and change point
`mart_forecast`	day: Jan–Jun actual, model, baseline, split; July forecast	model page, July baseline for actions
`mart_actions`	proposed action: cost, net benefit, range, action downtime, measurement	Q6 budget page

EDA and analysis plan

Start by reproducing the GM's naive bottom-10 (the DuckDB query in the guide), then show which of those 10 survive each adjustment. That before/after is our headline.

Q1. Truly underperforming machines (luck, location, or leak)

Build one row per machine with these decompositions, using days on floor only:

- **Luck:** luck residual = house win − theo. Outside factors (weather, traffic, time of month, events) change how much is played, but theo moves with play, so the residual should hold only spin-level luck. Test it two ways:
  - **CLT z-score:** (house win − theo) ÷ (σ × √(coin-in × avg bet)), where σ is the game's per-spin standard deviation from `game_math`. If `game_math` gives only a volatility label, rely on the bootstrap.
  - **Block bootstrap** (cross-check, and the fallback): resample 7-day blocks of each machine's days, keeping each day's residual and coin-in together, 10,000 times. Compare total residual ÷ total coin-in (ratio of sums), so busy days don't dominate. The middle 95% is the luck interval: including zero means luck can't be ruled out. It handles jackpots and needs no σ, but can look too narrow on a high-volatility machine that never hit a jackpot.
  - **Independence check first:** correlate each machine-day's scaled residual with weather, weekday, day of month, traffic and events. Near-zero correlations confirm the simple block bootstrap; report this in the Claims & Evidence appendix.
  - **Stratified version if a factor shows up:** resample weekend blocks only from weekends and weekday blocks only from weekdays (or by month, or by whichever factor correlated), so each resample keeps the real mix of day types.
  - **Multiple testing:** correct across ~300 machines with Benjamini–Hochberg, using p-values from the z-score or the bootstrap (share of resamples at or above zero), so about 15 chance flags at 5% don't become "findings."
- **Exposure / availability:** theo per available hour and coin-in per available hour. Availability = `available_hours` ÷ (`scheduled_seconds` ÷ 3600).
- **Location:** compare each machine to its zone/bank average for the same days (from `placements`). This also cancels floor-wide events, weather and traffic. Machines that moved are natural experiments: did coin-in follow the machine or the slot?
- **Faults while running:** compare theo per available hour in the 7 days before each fault ticket (`operations`) and after the repair. A bounce-back means the fault, not the machine, held it back.
- **Leak (payout settings):** observed hold vs. expected hold (1 − RTP), using the same significance tests on large coin-in. Join `game_math` by each day's game setting, since a machine's setting can change. Persistent gaps are inspection leads, never accusations.
- **Net contribution:** theo minus the machine's own costs (lease, service, power, repairs from `cost_rules` and the cost columns in `prepared_daily_view`), per available hour. Replacement candidates are ranked on this, not on theo alone.

- **Output:** a classification per machine: weak demand (true replace candidate), unlucky, held back by location, held back by downtime, faulty while running, or hold anomaly.

Q2. Demand changes across the floor

- Daily floor coin-in and theo from `property_day`, with weekday and holiday from `calendar_day`. Also show coin-in per machine-day, since 300–308 machines were on the floor on different days.
- **Season vs. trend:** six months alone can't separate the two. Compare our monthly pattern with the Missouri benchmark monthly totals and LVCVA visitation; if they moved too, the change is likely seasonal or citywide.
- **Visits vs. spend:** split carded demand into number of visits × play per visit using `trips` (visits to this casino only, not whole Las Vegas trips). Fewer visits and smaller visits call for different fixes.
- Overlay events and LVCVA monthly visitation (context only, not a model input).
- Zone-level coin-in vs. `zone_traffic_hourly` entries; drop offline counter hours via `counter_status`.
- **Defining "a change":** compare before vs. after a specific dated event, or run a simple change-point check on daily coin-in per machine-day.
- **Competing explanations to test:** citywide visitation and weather (NOAA `weather_daily`: heat, rain) before blaming anything on the floor.

Q3. Play lost to downtime

- **Lost theo, estimated from neighbors on the same hours:** lost theo ≈ neighbors' theo per available hour during the outage × (this machine's usual rate ÷ neighbors' usual rate) × outage hours. Neighbors felt the same event, weather and traffic, so the day's outside factors cancel out, and a six-month average never stands in for "normal."
- **"Usual rate" ratio:** computed from nearby weeks in the same placement and hour band, leaving out fault-ticket days, partial-availability days and the first days after an install or move.
- Use neighbors in the same zone but a different bank, so players who switch to the next machine during the outage don't inflate the estimate.
- **Test the method with fake outages:** pick hours when machines were working, pretend they were down, estimate their "lost" theo with the method, and compare to what they actually earned. Report the error (MAE) in the Claims & Evidence appendix and use it as the uncertainty range on lost-theo figures.
- **Net of switching:** some players move to the next machine during an outage. Check whether neighbors' play rises during outages and report net floor loss next to the gross figure.
- **Controllable vs. not:** split `operations` tickets into faults, planned maintenance, moves and installs. Only faults and slow repairs are fixable by a technician or preventive maintenance.
- **Response vs. repair time:** slow response points to staffing (add a technician); slow repair points to the machine (preventive maintenance).
- **Hidden downtime:** flag long zero-play stretches in `machine_hour` while a machine is marked available and its neighbors are busy. These may be unreported breakdowns.
- Break down by manufacturer, fault type and shift (`shift_roster`). Show when with a heatmap of lost theo by weekday × hour.

Q4. Tracked player outcomes

- From `player_period`: coin-in, payout, net, expected loss by tier, age band and home region.
- **Carded share:** report what share of total coin-in is carded (from `carded_*` columns in `machine_day`), so readers know how much of the floor these results describe. Reconcile with those columns, not total meters.
- **Expected vs. actual loss:** compare the two by segment to show how much of players' results was luck.
- **Concentration:** share of carded net win from the top 1% / 10% of players (Lorenz curve).
- **Share of winners:** state what share of players and visits came out ahead.

- **Protective angle:** a simple metric, such as players whose losses run far above their expected loss, used only for protective ideas, never to push more play.

Q5. Comp program

Compare what players spent with what the comp program cost. Theo is the house's expected win, so it is a modeled figure and not spend.

- **Comp cost vs. spend, daily and by tier × month:**
  - **Headline:** comp cost ÷ actual carded net loss (`player_period`), i.e. cents of comps per $1 players lost.
  - Comp cost ÷ carded coin-in: amount wagered, which is less luck-driven.
  - Comp cost ÷ total spend: actual loss plus F&B; and other receipts (`fnb_transactions`, aggregated before joining), if "spent" includes non-gaming.
  - Theo ÷ comp cost: secondary cross-check only.
  - All ratios are ratio of sums. Subtract comp cost once (it is inside cogs). Convert `loyalty_ledger` and receipt timestamps to gaming dates before daily totals.
- **Luck check:** actual loss swings with luck, especially in small tier-months. If comp cost ÷ actual loss jumps but comp cost ÷ coin-in doesn't, treat it as luck, not a comp change.
- **Change point:** run it on comp cost ÷ actual loss and on comp cost ÷ coin-in. Flag a change (rate change, tier rule, promotion) only when both move. Also check whether comp cost moved without a matching move in coin-in.
- **Do comps work?** Compare return visits and actual spend of comped players with similar uncomped players, matched on tier and prior coin-in. Treat as suggestive, not causal.
- **Unredeemed points:** points earned minus redeemed in `loyalty_ledger` are a future cost. Report the balance and how it moved.

Beyond the six: security incidents vs. specific banks or shifts, the casino's own survey vs. LVCVA, Missouri benchmark hold as a sanity check on our floor's hold.

Model plan

Forecast daily floor coin-in, train Jan 1–Apr 30, test May 1–Jun 30, and use the result to time technician coverage and promotion weekends. This is the guide's recommended route, so judges know exactly what to look for.

Rules to prevent leakage:

- Choose the model inside the training period. Fit both candidates on Jan–Mar, compare MAE on April, keep the winner, then refit it on all of Jan–Apr. Only that one model and the baseline ever see May–Jun, so the holdout stays a fair test.
- Freeze both forecasts (save predictions to a file with a timestamp) before computing any holdout error.
- Fit any scaling or averages on Jan–Apr only.
- Don't feed the `lag_*` columns from `prepared_daily_view` into a multi-day forecast across May–Jun. They're built from prior days' actuals, so they're only safe one day ahead.

	Baseline	**Model**
Method	Constant training-period daily mean	Two candidates, one chosen on April validation: weekday average, or a regression on weekday + holiday + scheduled event flags (or Holt-Winters weekly seasonality)
Inputs	None	Only what is known in advance: calendar, holidays, events announced before the date
Not allowed	LVCVA or survey data published later; holdout-day results feeding later predictions; next week's repairs	

	Baseline	Model
Metric	MAE in dollars on May–Jun	Same, plus MAE by weekday

If we add a machine-level model later (e.g. predicting faults), split by time the same way.

After validation: the July forecast. Refit the chosen method on all of Jan–Jun and forecast July 1–30, using only calendar, holiday and announced-event inputs. This forecast picks the promotion weekend and serves as the "no action" baseline for measuring the budget actions.

How it changes a decision: if the model beats the baseline mainly on Fri–Sat and event days, staff the added technician on those shifts and pick the promotion weekend on a soft weekend in the July forecast, not a busy one. If it doesn't beat the baseline, say so and fall back to weekday staffing; that still earns credit.

Dashboard page: actual vs. both forecasts over time with the train/test split marked, MAE table, residuals by weekday, and a one-line "so what".

Budget recommendation framework

Pick actions only after Q1–Q5 name the cause; each cause maps to one fix. Choose at most 3 action types and spend no more than $100,000 over July 1–30.

Finding from EDA	Candidate action	Cost basis
Bottom-10 mostly unlucky	Keep the budget (or most of it)	$0
Truly weak demand	Lease replacement (e.g. $3,380 with 14-day lead) over purchase ($22,000)	Lease formula from the guide
Good machines in dead slots	Move machines ($1,800 each) or a bank ($9,500); zone signs ($7,500)	Benefit = net contribution uplift to the zone rate × days left after lead time and move downtime
Downtime concentrated in one manufacturer or shift	Preventive maintenance ($9,000) or technician ($11,520, fixed)	Benefit = recovered net loss from Q3 (after players switch machines), in net contribution
Hold anomaly	Inspect and verify payout ($1,200 each, up to 15)	Benefit = information; frame as risk control
Comp cost drifted (comp cost per $ actual loss and per $ coin-in rose; per $ theo is not used)	Comp pilot with control group ($4,000), not with the floor-wide change	Include extra comp cost in budget

For every funded action, the dashboard and write-up show: cost (using `lead_days` and the guide's formula), expected effect in net contribution dollars, a low/high range, and the metric plus control group we'd watch in July.

Sensitivity: show the plan still pays off (or not) if the effect is half our estimate. Check `action_catalog` incompatibilities before finalizing: no move and replacement on the same machine. Also subtract each action's own downtime from the catalog (e.g. a machine is off the floor while it moves or is replaced) from its July benefit days.

Further rules for the plan:

- **Benefits in net terms:** estimate each action's benefit as net contribution (theo minus machine costs), the same measure Q1 ranks on.
- **Measure with theo, not house win.** Thirty days of house win is too noisy to judge an action. Before funding, check that the expected effect is larger than normal 30-day variation in theo for the targets.
- **Use the forecast as the July baseline.** The model's prediction of what July would look like without the action is the comparison for measuring it, which ties the model to a decision.
- **Survey option:** consider the bigger guest survey ($3,500) if Q2 or Q5 ends with an open "why."

Tableau dashboard design

Build it as a Tableau Story (or tabbed dashboards) that walks the manager from question to decision. Each page has one headline sentence stating its finding.

1. **The verdict:** GM's bottom-10 vs. our adjusted bottom-10, with each machine's reason (unlucky / location / downtime / faulty while running / hold / weak), labeled "Jan–Jun 2025" since the tests use the full period.
2. **Machine explorer:** scatter of actual vs. theo per available hour, colored by classification; click a machine for its daily history and placements.
3. **Floor demand:** daily coin-in with events and LVCVA context; zone map from `floor_locations` coordinates.
4. **Downtime:** lost theo by month, manufacturer, shift; weekday × hour heatmap.
5. **Players and comps:** outcomes by segment, concentration curve, comp cost per $ actual loss and per $ coin-in over time.
6. **Forecast:** model vs. baseline on the holdout, with MAE and the decision it drives.
7. **The $100K plan:** each action with cost, expected effect, range, measurement plan, linked to the page that justifies it.

The four judge tests, built in from day one:

- **Change the period:** a global date filter (applied to all worksheets using `gaming_date`) on every page that reads a daily or hourly mart. Because ratios must be ratio of sums, compute hold, availability and theo per available hour as Tableau calculated fields (`SUM([house_win]) / SUM([coin_in])`) over `mart_machine_daily`, not precomputed per-machine ratios. The Q1 classifications, z-scores and bootstrap intervals come from Python and don't recalculate in Tableau: the verdict page states they cover Jan–Jun, and a month selector reads `mart_machine_month` for shorter periods.
- **Drill into a bank or guest group:** bank and zone filters on machine pages; tier / age band / region filters on the player page.
- **Find the validation:** the forecast page states the split, the baseline and MAE in plain words.
- **Trace a recommendation:** action cards on page 7 link (dashboard actions) back to the evidence page.

Accessibility: colorblind-safe palette, labels not just colors, dollar units on every axis, tooltips that define terms (theo, hold).

Roles, timeline and Devpost checklist

Suggested split for a team of four; merge roles if you are fewer.

Role	Owns
Data engineer	DuckDB views, validation checks, all marts, `run_all.py`
Analyst A	Q1 scorecard and Q3 downtime
Analyst B	Q2 demand, Q4 players, Q5 comps; the forecast model
Dashboard + story lead	Tableau build, write-up, video, Claims & Evidence appendix

Phases, as a share of the hacking time (open question: confirm the exact deadline on the Devpost event page):

1. **First 10%:** everyone loads data, runs the guide's bottom-10 query, reads the data dictionary. Data engineer ships the validation script.
2. **Next 35%:** EDA per question in notebooks; first marts land; dashboard lead wireframes the 7 pages on fake data.
3. **Next 25%:** model trained and frozen; marts finalized; Tableau pages built on real marts; budget plan drafted.

4. **Next 20%:** fill the Claims & Evidence appendix as findings settle; run the four judge tests; publish to Tableau Public.
5. **Last 10%:** record the 2-minute video, cut the write-up to 400 words, test the link in a private window, submit.

Submission checklist:

[ ] Public Tableau Public link, opened in a private browser window
[ ] Video, 2 minutes or less, covering the dashboard and priced actions
[ ] Write-up, 400 words or less
[ ] Model vs. baseline on a time-based holdout, with MAE
[ ] All code and SQL (repo link)
[ ] Up to 3 priced action types with cost, effect, uncertainty, measurement plan
[ ] Claims & Evidence appendix: claim → table/query → result → alternative explanation → limitation
[ ] AI-use disclosure in the appendix
[ ] Note that the casino and data are fictional

Pitfalls checklist

The data engineer turns as many of these as possible into automated checks in the validation script.

- Ratios are ratio of sums, never the mean of daily ratios
- The 1,018 off-floor rows (empty `location_id`) are excluded
- Location comes from `placements` by date, since machines move
- Day length comes from `scheduled_seconds` (Mar 8 has 23 hours)
- Timestamps are converted to gaming date (6 a.m. to 6 a.m.) before daily grouping
- `_cents` and `_usd` columns are never mixed
- Negative house win days are kept: they're real luck, not errors to drop or fix
- Comp cost is subtracted once, not alongside `linked_pos_cogs_usd`
- Comp rates use actual loss and coin-in as the denominator, not theo (theo is expected win, not spend)
- Carded totals are compared to `carded_*` meters, not all play
- Tables are aggregated before joining sessions to `fnb_transactions`
- `trips` are treated as visits to this casino, not whole Las Vegas trips
- Missing traffic counts are treated as offline, not zero; entries are not unique visitors
- LVCVA figures describe Las Vegas, not this casino's guests
- `lag_*` columns are used only one day ahead, never in a multi-day holdout forecast
- Hold anomalies are described as leads to inspect, not wrongdoing
- No ideas that push heavy losers to gamble more